System Architecture – Practical Task

Part 1: Real-World Examples of Architecture Patterns

1. Layered Architecture

Real-world example: Online Banking System (e.g., Standard Bank's internet banking portal) (Standard Bank Group, n.d.)

An online banking platform is a classic layered system built around four tiers (Richards, 2015):

- **Presentation layer:** The web browser or mobile app that customers interact with – account dashboards, fund-transfer forms, and statement views.
- **Business layer:** The application server handling authentication, transfer validation, fraud-detection rules, and interest calculations.
- **Persistence layer:** Data-access objects and ORM (Object Relational Mapping) components that translate business objects into database queries.
- **Database layer:** A relational database (e.g., Oracle DB) storing customer accounts, transactions, and audit logs.

When it is most appropriate:
Layered architecture is best when a system has distinct, separable concerns – a visible front-end, significant business logic, and persistent data storage. It suits enterprise applications where teams are organised around specialisations (UI designers, backend developers, database administrators) and where regulatory compliance requires clear audit boundaries between tiers (Richards, 2015).

Why it is appropriate for this example:
Banking software demands strict separation of concerns for both security and maintainability. Keeping authentication and business rules in the business layer means a UI redesign never accidentally exposes database logic. The persistence layer abstracts database changes so the rest of the system is unaffected when the database is upgraded. Each layer can also be tested and certified independently, which is critical for financial-sector compliance requirements (Standard Bank Group, n.d.).

2. Repository Architecture

Real-world example: Integrated Development Environment (IDE) – JetBrains IntelliJ IDEA (JetBrains, n.d.)

An IDE such as IntelliJ stores all project artefacts (source files, build configurations, dependencies, run configurations) in a central workspace repository (JetBrains, n.d.). Independent tools – the code editor, debugger, compiler, static analyser, version-control plugin, and test runner – all read from and write to this shared repository rather than communicating directly with each other.

When it is most appropriate:
Repository architecture is best when multiple independent components need to share large amounts of long-lived data, and when those components should remain loosely coupled to each other. It suits scenarios where data is the primary asset (e.g., development tools, data-warehouse systems, content management platforms) and where a single source of truth must be maintained (Richards, 2015).

Why it is appropriate for this example:
An IDE integrates many tools that all operate on the same codebase. If each tool communicated point-to-point with every other tool the coupling would be unmanageable. The repository pattern keeps all tools decoupled: the debugger does not need to know about the static analyser – both simply read the source files from the shared project store. Adding a new plugin (e.g., a Docker integration) only requires it to interact with the existing repository, not with every existing tool (JetBrains, n.d.).

3. Client-Server Architecture

Real-world example: Gmail (Google's email service) (Google, n.d.)

Gmail is a textbook client-server system (Google, n.d.). The clients (web browser, Android/iOS app, desktop email clients via IMAP) connect over the internet to Google's distributed mail servers. The servers host the services – email storage, search indexing, spam filtering, and calendar integration – which all clients access uniformly regardless of the device they are using.

When it is most appropriate:
Client-server architecture is best when many users need concurrent access to shared services or data, when centralised management of data is required, or when clients are geographically dispersed. It suits any system where the server-side resources (compute, storage) are too large to reside on each client device (Richards, 2015).

Why it is appropriate for this example:
Email is inherently a shared, network-dependent service. Users expect to access the same inbox from a phone, laptop, and tablet simultaneously. Centralising mail on servers allows Google to apply consistent spam filters, enforce storage quotas, and ensure that a message sent from one device immediately appears on all others. A peer-to-peer approach would make this consistency extremely difficult and would not scale to billions of users (Google, n.d.).

4. Pipe and Filter Architecture

Real-world example: Data ETL Pipeline – Apache Kafka and Apache Spark, used by companies such as LinkedIn and Uber (Apache Software Foundation, n.d.a; Apache Software Foundation, n.d.b)

In a big-data ETL (Extract, Transform, Load) pipeline, raw event data flows through a series of independent processing stages (filters) connected by data channels (pipes) (Apache Software Foundation, n.d.a):

- **Ingest filter:** Apache Kafka ingests raw event streams (e.g., user ride requests, GPS coordinates).
- **Cleanse filter:** A Spark job removes duplicate records and normalises field formats.
- **Enrich filter:** Another Spark stage joins the stream with reference data (e.g., mapping coordinates to city zones).
- **Aggregate filter:** Results are aggregated into summary metrics (average ride time per zone per hour).
- **Load filter:** Final data is written to a data warehouse (e.g., Amazon Redshift) for analytics dashboards.

When it is most appropriate:
Pipe and filter is best when data must pass through a well-defined sequence of independent transformations, when processing steps are reusable across different pipelines, and when the volume of data makes parallel or streaming execution desirable. It is particularly suitable for batch processing, log analysis, video transcoding, and any data-driven business workflow (Apache Software Foundation, n.d.b).

Why it is appropriate for this example:
ETL pipelines are inherently sequential transformation problems. Each stage has a single responsibility, making the system easy to test, extend, or replace one filter without affecting others. A new data quality check can be inserted as an additional filter without redesigning the entire pipeline. The pattern also maps naturally onto distributed frameworks like Spark, which run filters in parallel across a cluster to handle massive data volumes efficiently (Apache Software Foundation, n.d.a; Apache Software Foundation, n.d.b).

Part 2: Systems Combining Two or More Patterns

Example 1: Netflix – Microservices Architecture combined with Pipe and Filter

Description:
Netflix uses a microservices architecture where independent services handle specific functions: user authentication, content recommendations, billing, streaming delivery, and search (Fowler and Lewis, 2014; Netflix, n.d.). Alongside this, Netflix operates a pipe and filter pipeline for video encoding. When a new title is uploaded, the raw video file flows through a series of filters – format validation, transcoding into multiple resolutions (4K, 1080p, 720p, 480p), audio normalisation, subtitle embedding, and packaging into streaming formats (HLS, MPEG-DASH) – before being distributed to edge servers globally (Netflix, n.d.).

Strengths of the combined system:

- **Independent scalability:** Each microservice scales based on its own demand (e.g., the recommendation service can scale independently of the billing service during peak viewing hours). The encoding pipeline stages can also be parallelised to process many videos simultaneously.
- **Fault isolation:** A failure in the subtitle encoding filter does not bring down the recommendation microservice. Services and pipeline stages fail in isolation.
- **Specialised optimisation:** Each microservice and each filter can be written in the language and framework best suited to its task.
- **Continuous delivery:** Microservices can be updated and deployed independently, enabling Netflix to push hundreds of deployments per day without system-wide downtime (Fowler and Lewis, 2014).

Limitations of the combined system:

- **Operational complexity:** Managing hundreds of microservices and encoding pipeline instances requires sophisticated orchestration tools (Netflix uses Kubernetes and custom tooling), adding infrastructure overhead (Netflix, n.d.).
- **Latency accumulation:** Both patterns introduce inter-component communication costs – network calls between microservices and data hand-offs between pipeline filters. These latencies can compound under high load.
- **Difficult debugging:** Tracing a fault across multiple microservices and pipeline stages requires distributed tracing tools (e.g., Zipkin). Without them, diagnosing failures is very difficult.
- **Data consistency challenges:** With many independent microservices each owning their own data store, maintaining consistency across services requires careful design (e.g., eventual consistency, sagas) (Fowler and Lewis, 2014).

Example 2: SAP ERP System – Layered Architecture combined with Repository Architecture

Description:
SAP's Enterprise Resource Planning (ERP) system combines a layered architecture with a repository pattern (SAP SE, n.d.). The system is structured in layers: a presentation layer (SAP Fiori web UI / GUI), a business logic layer (ABAP application server processing HR, finance, procurement, and supply-chain rules), a persistence layer (SAP's database abstraction), and a database layer (SAP HANA in-memory database). Cutting across all these layers is a central data repository – a unified enterprise database that stores all business data shared across modules. Every module (HR, Finance, Inventory, Procurement) reads from and writes to this central repository, ensuring a single source of truth for the entire organisation (SAP SE, n.d.).

Strengths of the combined system:

- **Single source of truth:** All departments access the same data. When the finance module records a purchase, the inventory module immediately reflects the updated stock – eliminating data silos and reconciliation errors (SAP SE, n.d.).
- **Clear separation of concerns:** The layered structure keeps UI, business rules, and data access

logically separate. A change to a UI screen does not affect financial calculation logic.

- **Consistency and integrity:** Centralising data in the repository makes it straightforward to enforce referential integrity and transactional consistency across the entire enterprise.
- **Reusability:** The layered design allows individual business-logic components to be reused across different modules (e.g., a currency-conversion function used by both Finance and Procurement) (Richards, 2015).

Limitations of the combined system:

- **Single point of failure:** The central repository is the backbone of the entire system. A database outage affects every module simultaneously – HR, Finance, Logistics – potentially halting business operations (SAP SE, n.d.).
- **Performance bottlenecks:** All modules compete for access to the central database. During month-end financial closing, for example, heavy concurrent access can degrade performance across all departments.
- **Scalability constraints:** The tightly coupled layered structure makes it difficult to scale individual modules independently. Scaling the inventory module typically means scaling the entire application stack.
- **High implementation complexity and cost:** Configuring a combined layered/repository ERP system to match an organisation's specific processes requires significant expertise, time, and licensing cost – as evidenced by the notoriously long and expensive SAP implementation projects many enterprises undertake (SAP SE, n.d.).

Reference List

Apache Software Foundation (n.d.a). *Apache Kafka documentation*. Apache Software Foundation. Available at: https://kafka.apache.org/documentation/ [Accessed 21 April 2026].

Apache Software Foundation (n.d.b). *Apache Spark documentation*. Apache Software Foundation. Available at: https://spark.apache.org/docs/latest/ [Accessed 21 April 2026].

Fowler, M. and Lewis, J. (2014). *Microservices*. Available at: https://martinfowler.com/articles/microservices.html [Accessed 21 April 2026].

Google (n.d.). *Gmail*. Google LLC. Available at: https://www.google.com/gmail/ [Accessed 21 April 2026].

JetBrains (n.d.). *IntelliJ IDEA – the IDE for professional development in Java and Kotlin*. JetBrains s.r.o. Available at: https://www.jetbrains.com/idea/ [Accessed 21 April 2026].

Netflix (n.d.). *Netflix Tech Blog*. Netflix, Inc. Available at: https://netflixtechblog.com/ [Accessed 21 April 2026].

Richards, M. (2015). *Software architecture patterns* (1st ed., pp. 1–8). O'Reilly Media, Inc.

SAP SE (n.d.). *What is ERP?* SAP SE. Available at: https://www.sap.com/products/erp/what-is-erp.html [Accessed 21 April 2026].

Standard Bank Group (n.d.). *Digital banking*. Standard Bank Group. Available at: https://www.standardbank.com [Accessed 21 April 2026].